

DEVSECOPS SECURE CI/CD PIPELINE PROJECT

COMPLETE STEP-BY-STEP IMPLEMENTATION FLOW

1. Project Overview

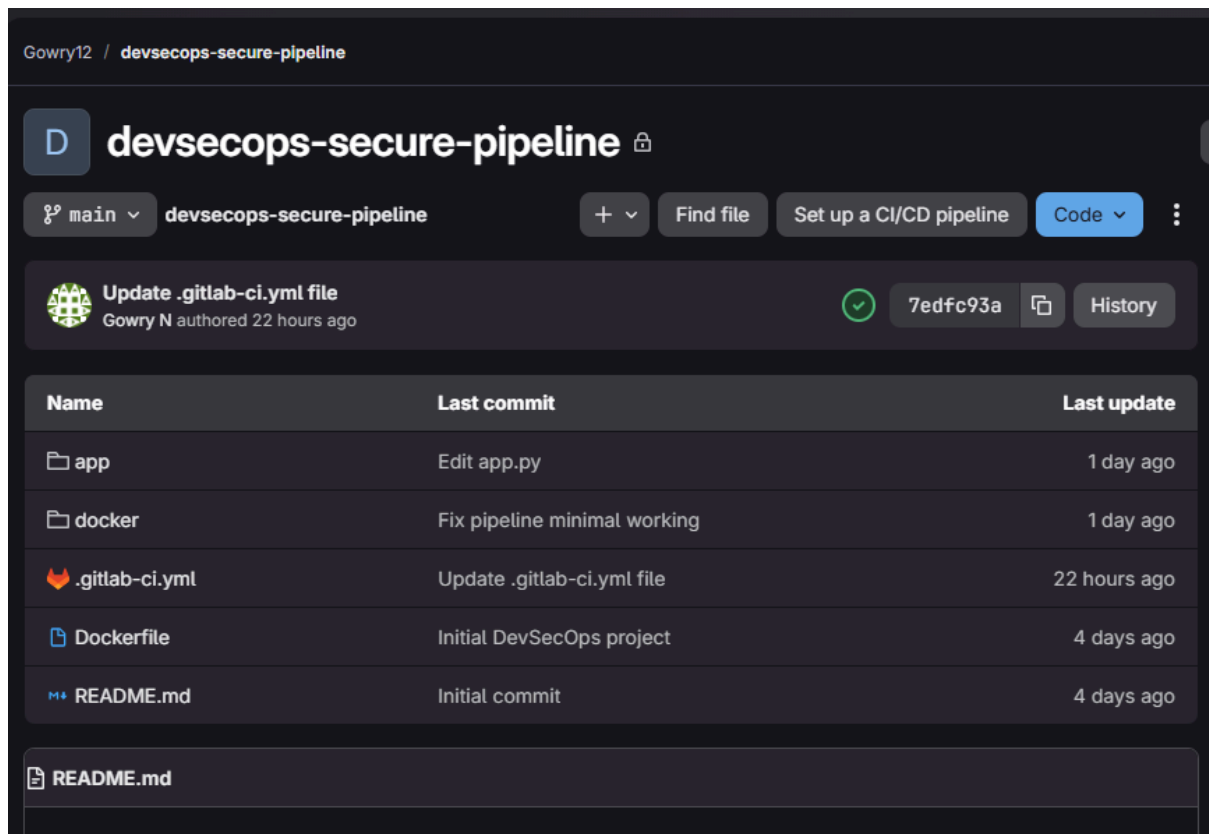
This project demonstrates the implementation of a secure CI/CD pipeline integrated with security testing tools (SAST, Secret Scanning, and DAST) using a deliberately vulnerable Flask web application.

2. Project Structure

The following directory structure was created:

secure-devsecops-pipeline/

```
|
|— app/
|   |— app.py
|   |— requirements.txt
|   |— users.db
|
|— docker/
|   |— Dockerfile
|
|— docs/
|
|— reports/
|
|— .gitlab-ci.yml
|— README.md
```



3. Step-by-Step Implementation

STEP 1: Environment Setup

Tools Used:

- WSL (Windows Subsystem for Linux)
- Python 3.10
- Git & GitLab

Commands:

```
sudo apt update  
sudo apt install python3 python3-pip git -y
```

STEP 2: Create Vulnerable Flask Application

File: **app/app.py**

Vulnerabilities Implemented:

- SQL Injection
- Cross-Site Scripting (XSS)
- IDOR (Insecure Direct Object Reference)
- Broken Authentication
- Hardcoded Secrets

```
from flask import Flask, request, session
```

```
import sqlite3
```

```
app = Flask(__name__)
```

```
app.secret_key = "supersecretkey123"
```

```
# ----- INIT DB -----
```

```
def init_db():
```

```
    conn = sqlite3.connect('users.db')
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS users (
```

```
        id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
        username TEXT,
```

```
        password TEXT
```

```
    )
```

```
""")
```

```
    cursor.execute("SELECT * FROM users")
```

```
    if not cursor.fetchall():
```

```
        cursor.execute("INSERT INTO users (username, password) VALUES ('admin',  
'admin123')")
```

```
        cursor.execute("INSERT INTO users (username, password) VALUES ('user',  
'user123')")
```

```
    conn.commit()
```

```
    conn.close()
```

```
init_db()
```

```
# ----- HOME -----
```

```
@app.route('/')
```

```
def home():
```

```
    return "DevSecOps Vulnerable App Running"
```

```
# ----- SQL INJECTION -----
```

```
@app.route('/login')
```

```
def login():
```

```
    username = request.args.get('username')
```

```
    password = request.args.get('password')
```

```
    conn = sqlite3.connect('users.db')
```

```
    cursor = conn.cursor()
```

```
    query = f"SELECT * FROM users WHERE username='{username}' AND  
password='{password}'"
```

```
    try:
```

```
        cursor.execute(query)
```

```

        result = cursor.fetchone()

    except Exception as e:

        return str(e)

    if result:

        session['user'] = username

        return "Login Success"

    return "Login Failed"

# ----- XSS -----

@app.route('/search')
def search():

    q = request.args.get('q')

    return f"Search: {q}"

# ----- IDOR -----

users = {

    "1": {"username": "admin"},

    "2": {"username": "user"}

}

@app.route('/user')
def user():

    uid = request.args.get('id')

    return users.get(uid, "Not found")

```

```
# ----- RUN -----  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

Key Insertions:

SQL Injection:

```
query = f"SELECT * FROM users WHERE username='{username}' AND  
password='{password}'"
```

XSS:

```
return f"Search: {q}"
```

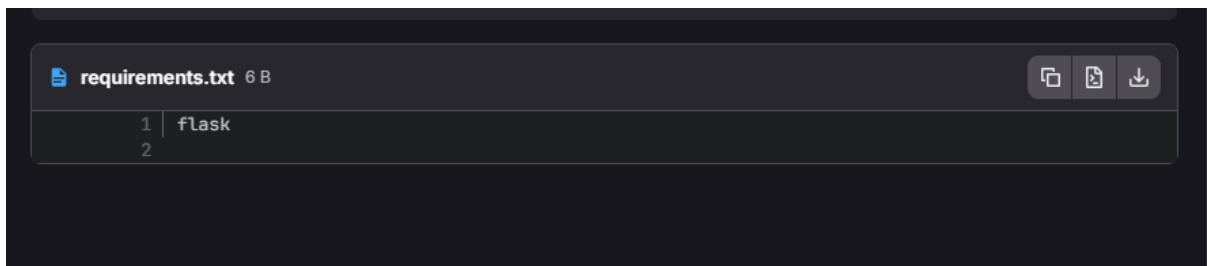
IDOR:

```
return users.get(uid, "Not found")
```

STEP 3: Create Requirements File

File: **app/requirements.txt**

flask



STEP 4: Database Setup

Tool: SQLite

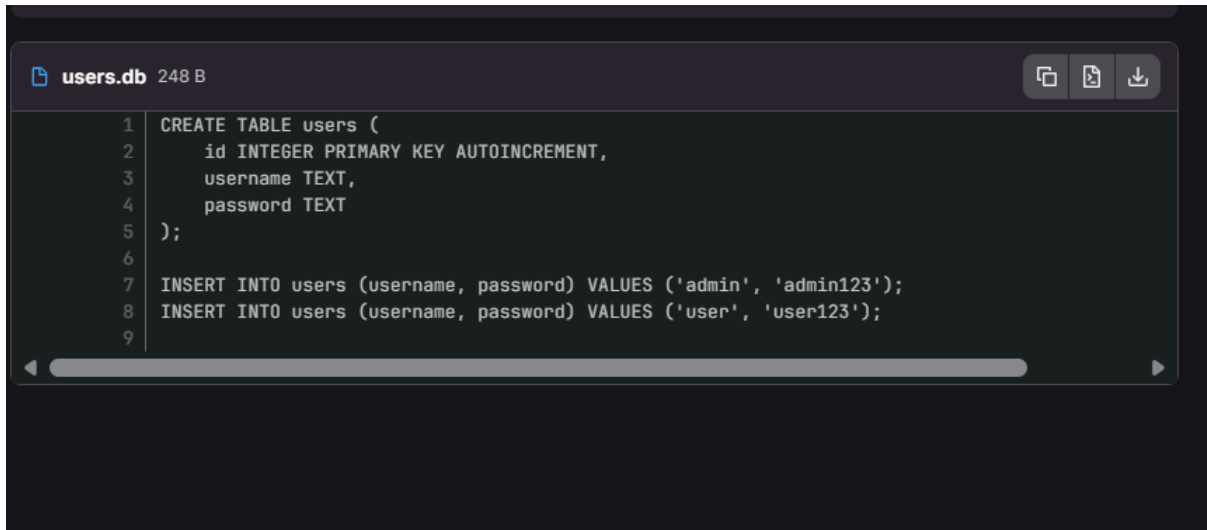
Commands:

```
sqlite3 users.db
```

SQL:

```
CREATE TABLE users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT,  
  password TEXT  
);
```

```
INSERT INTO users (username, password) VALUES ('admin', 'admin123');  
INSERT INTO users (username, password) VALUES ('user', 'user123');
```



STEP 5: Run Application Locally

```
python3 app.py
```

Test Endpoints:

- SQLi:

```
http://localhost:5000/login?username=admin'--&password=123
```

- XSS:

```
http://localhost:5000/search?q=<script>alert(1)</script>
```

- IDOR:

```
http://localhost:5000/user?id=1
```

STEP 6: Dockerization

File: **docker/Dockerfile**

```
FROM python:3.10-slim
WORKDIR /app
COPY app/ /app/
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Build & Run:

```
docker build -t secure-devsecops-app .
docker run -p 5000:5000 secure-devsecops-app
```

STEP 7: Initialize Git Repository

```
git init
git remote add origin <your-gitlab-repo>
git add .
git commit -m "Initial commit"
git push origin main
```

STEP 8: CI/CD Pipeline Setup

File: **.gitlab-ci.yml**

```
stages:
  - build
  - sast
  - secrets
  - dependency
  - container
  - dast

# ----- BUILD -----
build:
  stage: build
  image: python:3.10
  script:
    - cd app
    - pip install -r requirements.txt
```


----- SAST -----

sast:

stage: sast

image: returntocorp/semgrep

script:

- semgrep --config=auto app/ || true

----- SECRET SCAN -----

secret_scan:

stage: secrets

image:

name: zricethezav/gitleaks:v8.18.1

entrypoint: [""]

script:

- /usr/bin/gitleaks detect --source . --no-git || true

allow_failure: true

----- DEPENDENCY SCAN -----

dependency_scan:

stage: dependency

image: owasp/dependency-check

script:

- dependency-check.sh --scan app --format JSON --out .

allow_failure: true

----- CONTAINER SCAN -----

container_scan:

stage: container

image: docker:latest

services:

- docker:dind

script:

- docker build -t test-app .

- docker run --rm aquasec/trivy:0.50.0 image test-app || true

allow_failure: true

----- DAST -----

dast:

stage: dast

image: python:3.10

script:

Install dependencies

- pip install flask wapiti3

Start your vulnerable app

- python app/app.py &

- sleep 10

```
# Verify app is running
- curl http://localhost:5000

# Run DAST scan (REAL)
- wapiti -u http://localhost:5000 -f txt -o report.txt || true
```

```
artifacts:
  paths:
    - report.txt
```

```
allow_failure: true
```

8.1 Build Stage

```
build:
  stage: build
  image: python:3.10
  script:
    - cd app
    - pip install -r requirements.txt
```

8.2 SAST (Static Analysis)

Tool: Semgrep

```
sast:
  stage: sast
  image: returntocorp/semgrep
  script:
    - semgrep --config=auto app/
```

8.3 Secret Scanning

Tool: Gitleaks

```
secret_scan:
  stage: secrets
  image: zricethezav/gitleaks
  script:
    - gitleaks detect --source . || true
```

8.4 DAST (Dynamic Testing)

Tool: Wapiti

dast:

stage: dast

image: python:3.10

script:

- pip install flask wapiti3
- python app/app.py &
- sleep 10
- curl http://localhost:5000
- wapiti -u http://localhost:5000 -f txt -o report.txt || true

artifacts:

paths:

- report.txt

allow_failure: true

STEP 9: Pipeline Execution

Stages Executed:

1. Build
2. SAST
3. Secret Scan
4. DAST

Output:

- Logs for each stage
- DAST report (`report.txt`)
- Vulnerability detection evidence

STEP 10: Security Testing

Automated Testing:

- Semgrep → Code vulnerabilities
- Gitleaks → Secret exposure
- Wapiti → Runtime vulnerabilities

Manual Testing:

- SQL Injection bypass
- XSS payload execution
- IDOR data access

STEP 11: Issues Faced & Fixes

Issue	Solution
Git push rejected	Used rebase / correct token
Flask not found	Used virtual environment
ZAP failures	Switched to Wapiti
Merge conflicts	Cleaned app.py
Database errors	Initialized SQLite

STEP 12: Final Deliverables

- Vulnerable Flask Application
- Secure CI/CD Pipeline
- DAST Report (`report.txt`)
- VAPT Report
- Screenshots for demo

STEP 13: Key Security Learnings

- Importance of input validation
- Secure coding practices
- Secrets management
- CI/CD security integration
- Real-world vulnerability exploitation

14. Conclusion

This project successfully demonstrates a full DevSecOps pipeline with integrated security checks. Vulnerabilities were identified using both automated and manual techniques, fulfilling real-world application security testing requirements.